Dikshya Singh Shah
Class: 1205hf25045

APPENDIX 7-TEST REPORT

Dixen: Event Management, Discovery & Ticketing System



DIKSHYA SINGH SHAH

CLASS:1205hf2601per

18. FEB 2026

TECHNICAL EDUCATION COPENHAGEN

Guldblommevej 5, Ballerup

Dikshya Singh Shah

Class: 1205hf25045

INDHOLDSFORTEGNELSE

Overview 3

Automated Testing approach 3

Testing Result Summary 3

Repository Tests (using EF Core InMemory): 3

Services (using moq) 4

Controller Tests (using Moq): 5

Manual Testing 7

Example of Unit Testing 7

Reason for Technology Choices 7

Limitations and Future Work 8

Dikshya Singh Shah

Class: 1205hf25045

OVERVIEW

This test report documents the automated and manual testing performed on the Dixen web application. I, as the sole developer and tester, conducted the testing process.

A comprehensive suite of nearly 93 automated unit tests was implemented across the Repository, Controller, and the Service layers. Due to time constraints, full test coverage of the Service layer and integration-level testing could not be completed. However, the implemented tests cover the core business logic and data access functionality of the application.

Further expansion of service-layer testing and full integration testing is planned for future development to improve overall coverage and system robustness.

AUTOMATED TESTING APPROACH

Repository Tests: Used an InMemory database to simulate the real database environment for testing data operations reliably and efficiently.

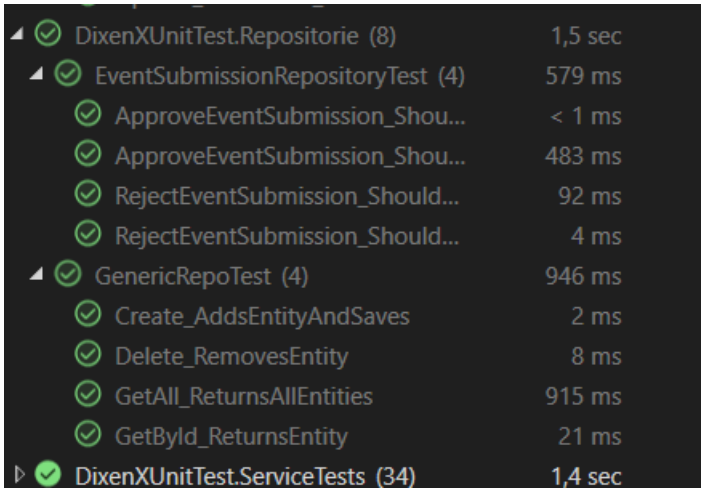
Service Tests: Tested core business logic by mocking repository dependencies to ensure correct processing, validation, and data handling within the service layer.

Controller Tests: Used the Moq framework to mock repository dependencies and test controller logic in isolation.

These choices allowed testing of core application logic without external dependencies, enabling fast and maintainable tests. *For more detail see Appendix 8 Source-Code)*

TESTING RESULT SUMMARY

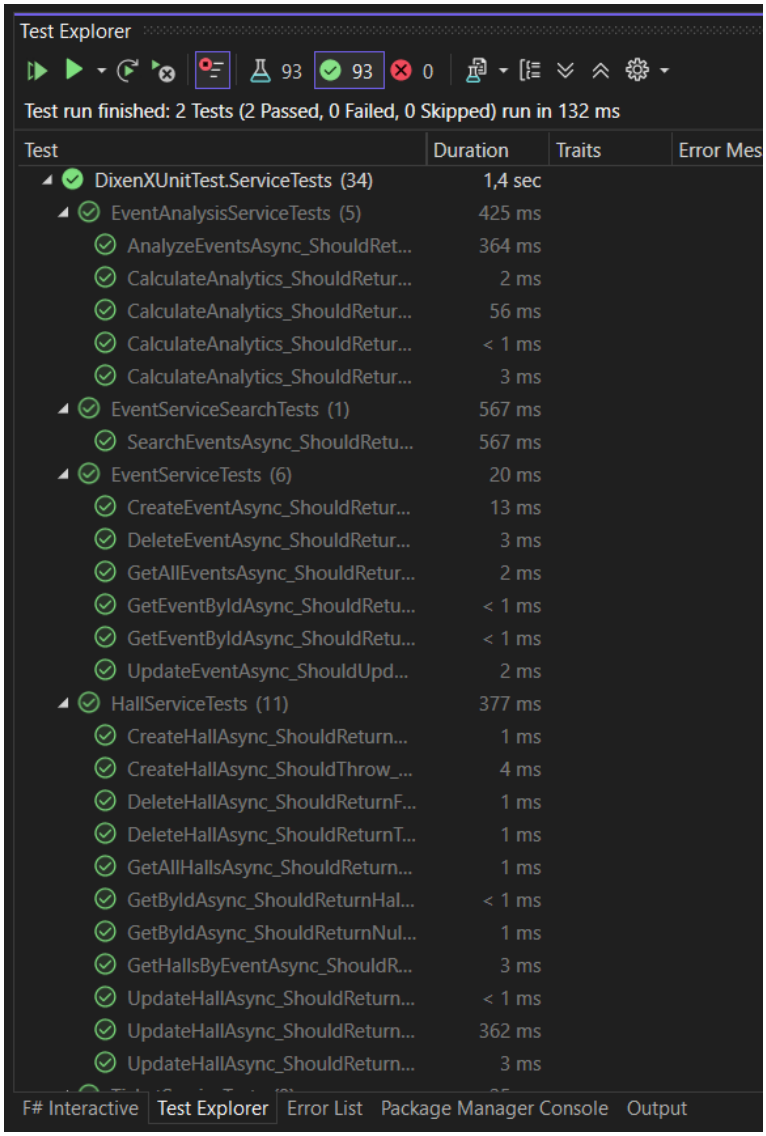
REPOSITORY TESTS (USING EF CORE INMEMORY):

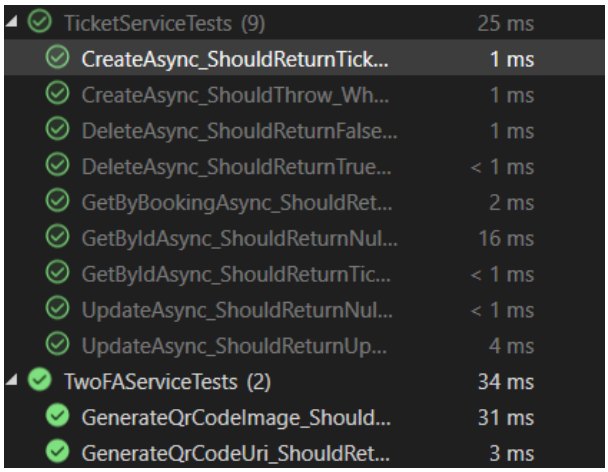


Dikshya Singh Shah

Class: 1205hf25045

SERVICES (USING MOQ)

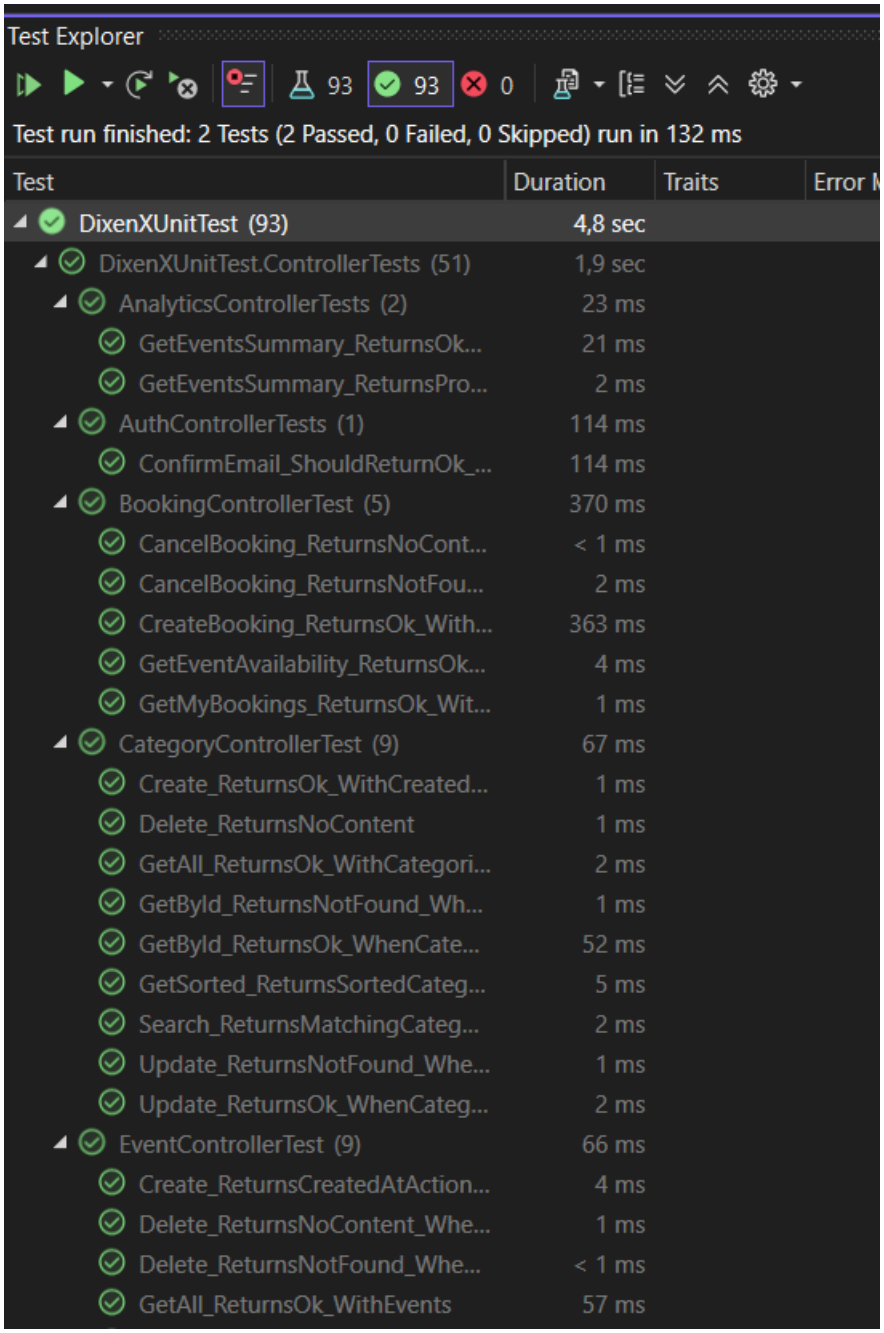




Dikshya Singh Shah

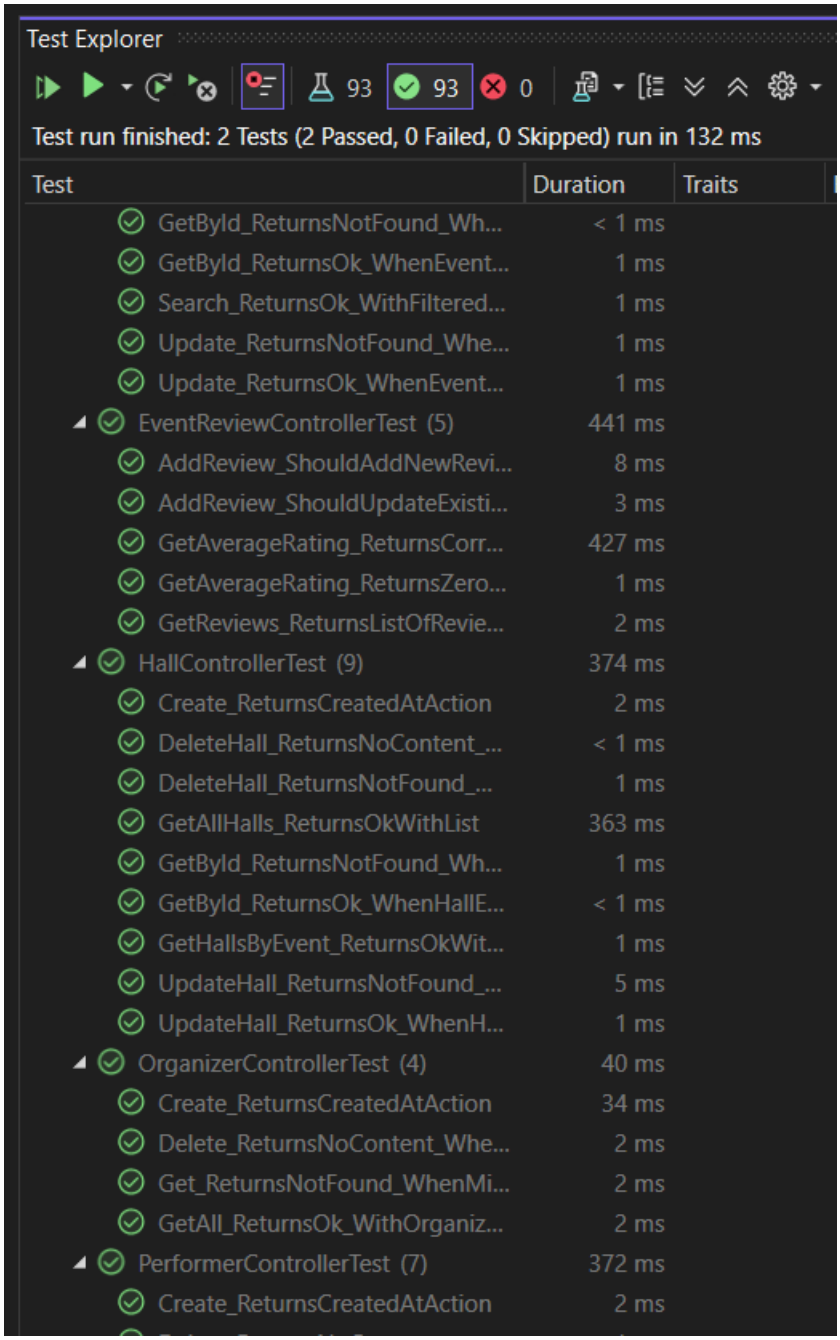
Class: 1205hf25045

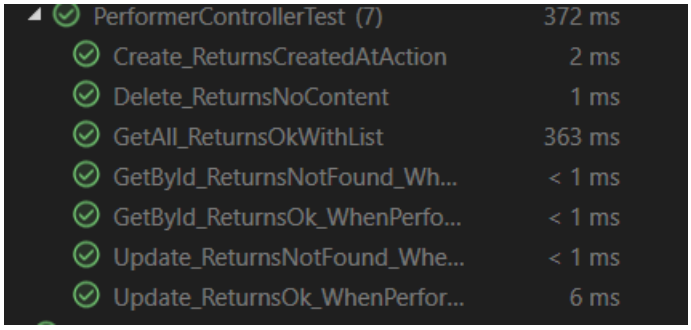
CONTROLLER TESTS (USING MOQ):



Dikshya Singh Shah

Class: 1205hf25045





MANUAL TESTING

Beyond automated tests, I also performed manual testing for:

- API Endpoints: Verified using tools like Swagger UI.
- Frontend UI: Inspected and tested functionality using browser developer tools to ensure buttons, forms, and navigation work as expected.

Manual tests helped identify issues not covered by automated tests and verified overall user experience.

EXAMPLE OF UNIT TESTING

Our API uses the Repository Pattern, with clear separation into Controllers, Services, and Repositories.

Example: Testing GetAll() in CategoryController

- Arrange: Create a list of dummy categories and configure the mocked repository to return this list.
- Act: Call the GetAll() method on the controller.
- Assert: Verify that the method returns the expected data and HTTP 200 response.

By using Moq, we simulate repository behavior, making the test independent from any database.

REASON FOR TECHNOLOGY CHOICES

EF Core InMemory Database:

- Allows us to simulate a real database environment without installing or configuring an actual database.
- Tests run fast because the database runs in memory and can be reset for each test.
- Provides realistic behavior for Entity Framework, ensuring repository logic is tested reliably.

Moq Library:

- Used to mock dependencies such as repository interfaces in controllers, allowing isolated testing of controller logic.

- Enables simulation of different scenarios (e.g., data found, data missing) without interacting with a real database.
- Keeps tests fast and focused on controller behavior.

The combination of In-Memory for repository testing and Moq for service and controller testing provides a balanced and robust test strategy, covering realistic data handling and isolated business logic

LIMITATIONS AND FUTURE WORK

Due to time constraints, full test coverage across all service methods and integration scenarios could not be completed.

In certain simple cases, minor data mapping or straightforward logic was implemented directly within controllers instead of introducing additional service abstractions. This decision was made to ensure timely project completion while maintaining functionality.

Future improvements include increasing service-layer test coverage, refactoring controller-level logic into services where appropriate, and implementing integration and UI-level automated tests.